

***Multi – level Linked List – Modified Insert After Element
(Main List and Child) – Program Analysis***

Program: –

```
Node *searchNode(Node *current, int target)
{
    if (current == nullptr)
        return nullptr;

    if (current->data == target)
        return current;

    Node *foundInChild = searchNode(current->child, target);
    if (foundInChild != nullptr)
    {
        return foundInChild;
    }
    return searchNode(current->next, target);
}

void insertAfterElement()
{
    if (head == nullptr)
    {
        cout << "List is empty. Please insert an element first." << endl;
        return;
    }

    int target, data, type;
    cout << "Enter the element to search for: ";
    cin >> target;

    Node *temp = searchNode(head, target);

    if (temp == nullptr)
    {
```

```

        cout << "Element " << target << " not found anywhere
in the list." << endl;
        return;
    }

    cout << "Enter data to insert: ";
    cin >> data;
    Node *newNode = createNode(data);

    cout << "Insert as (1) Next Node or (2) Child Node? ";
    cin >> type;

    if (type == 1)
    {
        newNode->next = temp->next;
        temp->next = newNode;
        cout << "Inserted " << data << " as NEXT of " <<
target << endl;
    }
    else if (type == 2)
    {
        newNode->next = temp->child;
        temp->child = newNode;
        cout << "Inserted " << data << " as CHILD of " <<
target << endl;
    }
    else
    {
        cout << "Invalid option." << endl;
        free(newNode);
    }
}

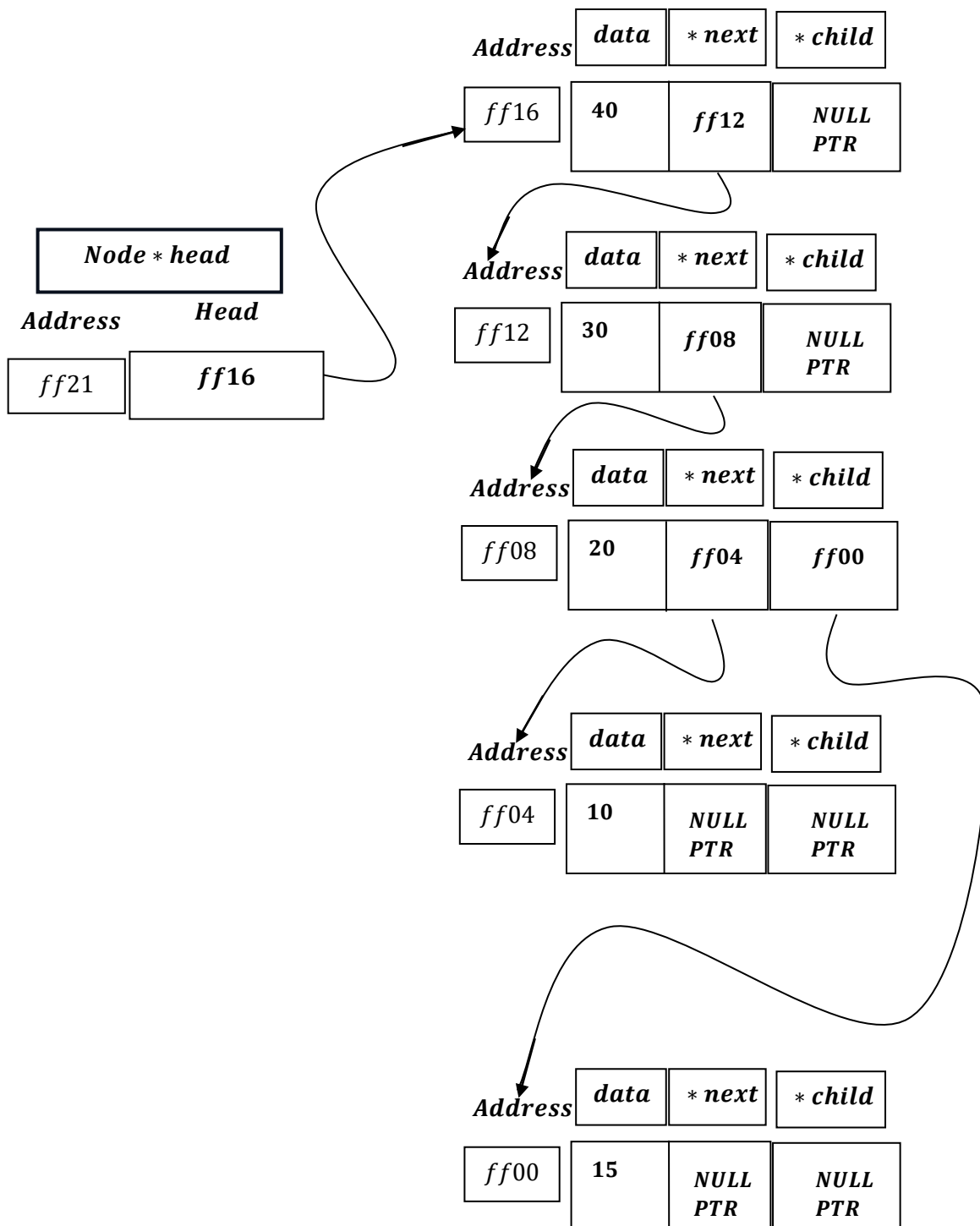
int main()
{
    int choice;
    while (true)
    {

```

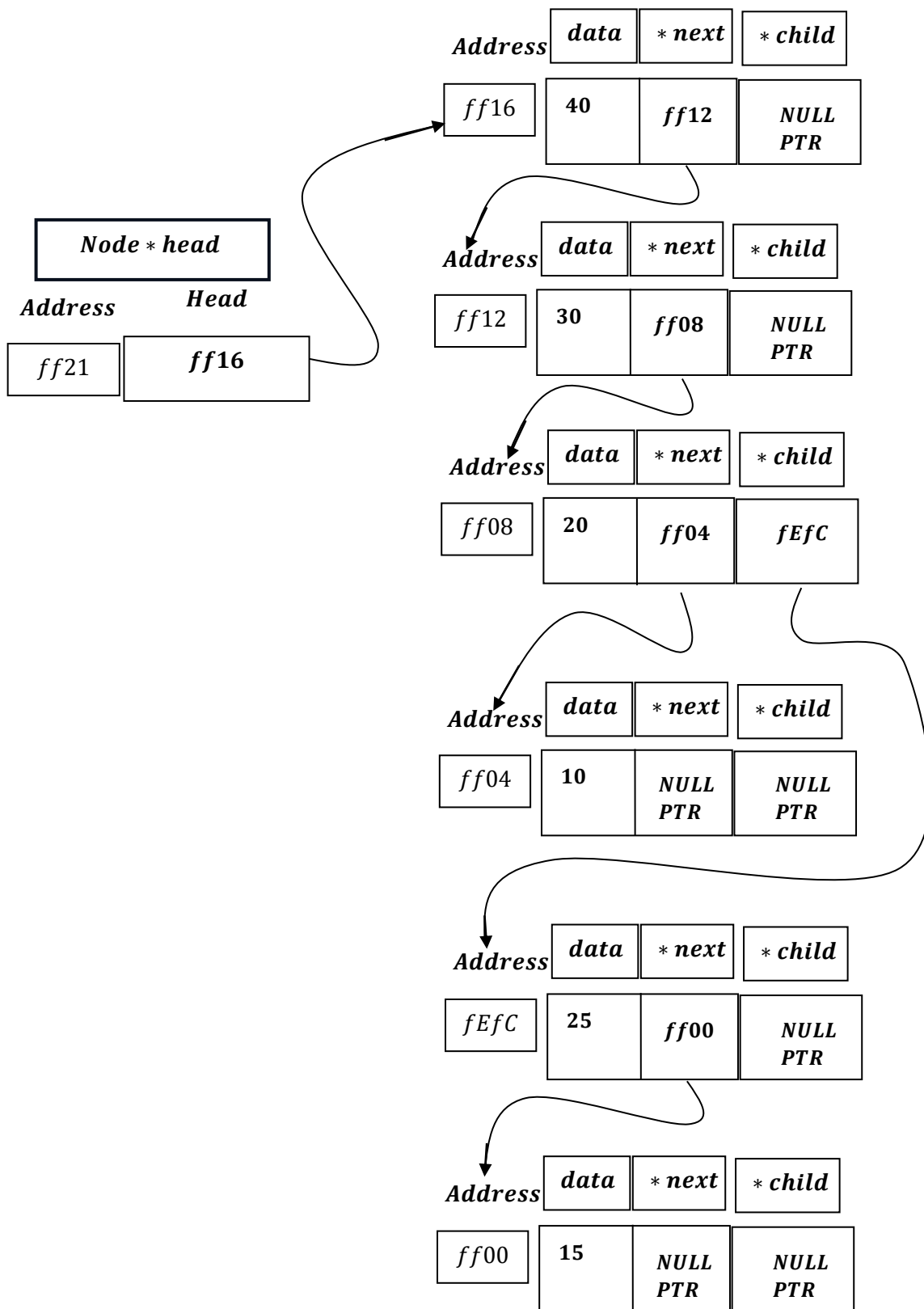
```
        cout << "4. Insert After Element (Link or Child)" <<
endl;
    switch (choice)
    {
    case 4:
        insertAfterElement();
    default:
        cout << "Invalid choice" << endl;
    }
}
return 0;
}
```

Program Analysis:

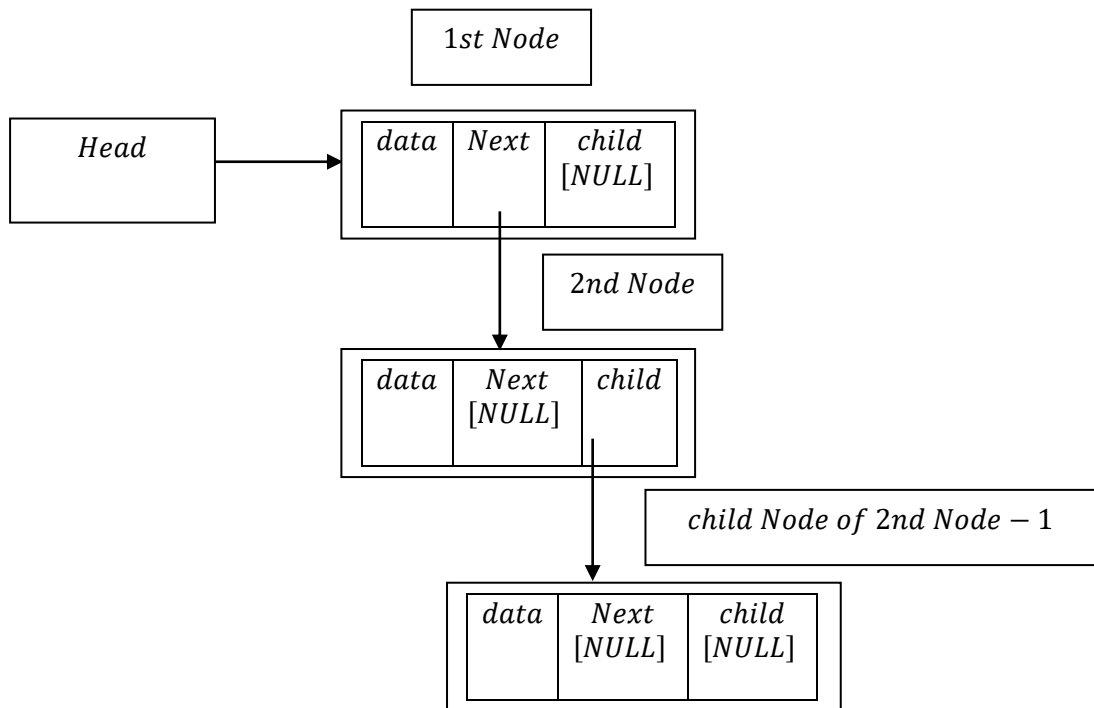
We had earlier :



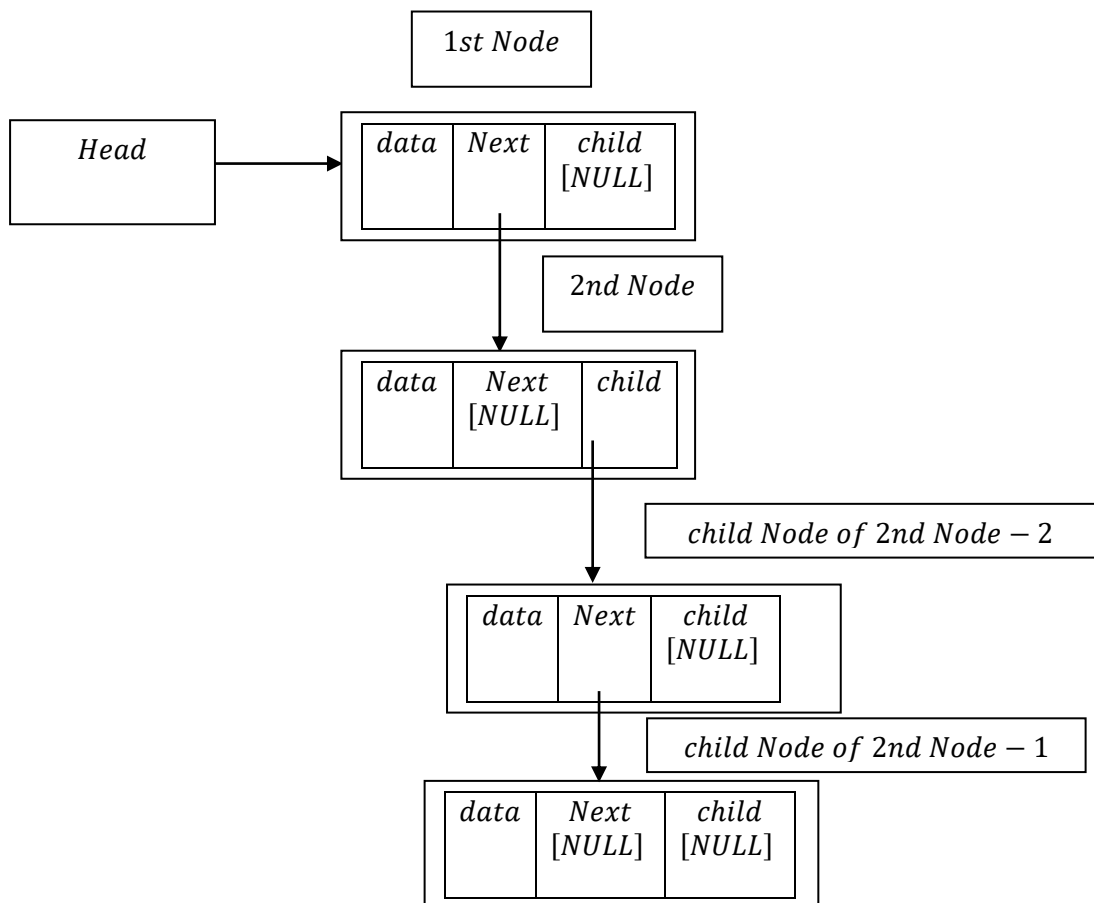
Next again if we insert child in address: ff08, we get:



i. e. Memory : fEfC goes up of ff00



Again if we enter another child Node of 2nd Node, we get :



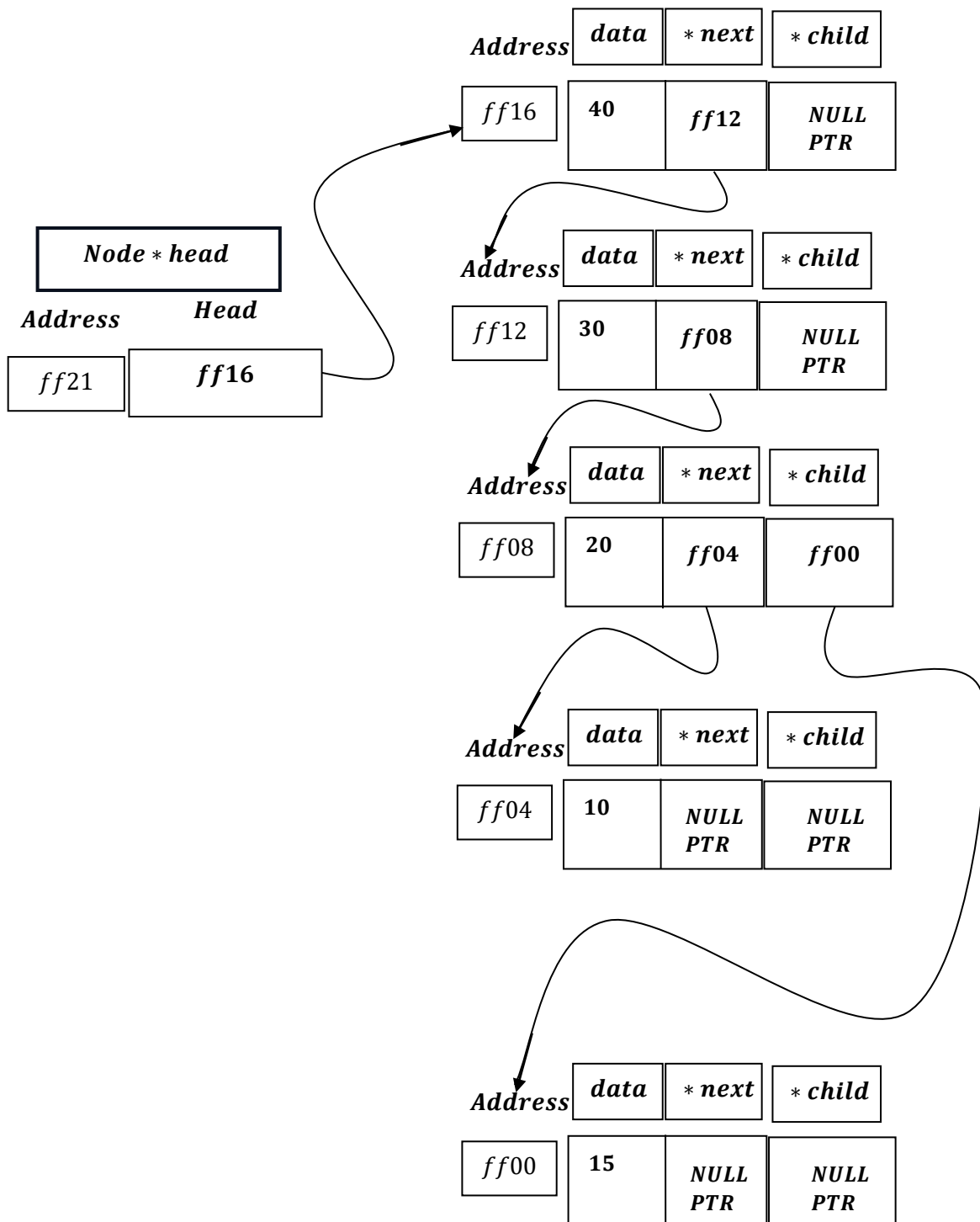
But if we want to enter child of child node ? There is no option, hence :

```
Node *searchNode(Node *current, int target)
{
    if (current == nullptr)
        return nullptr;

    if (current->data == target)
        return current;

    Node *foundInChild = searchNode(current->child, target);
    if (foundInChild != nullptr)
    {
        return foundInChild;
    }
    return searchNode(current->next, target);
}
```

Lets say we have :



And we want to enter child at data := 15.

```
cout << "Enter the element to search for: ";  
cin >> target;
```

target: 15

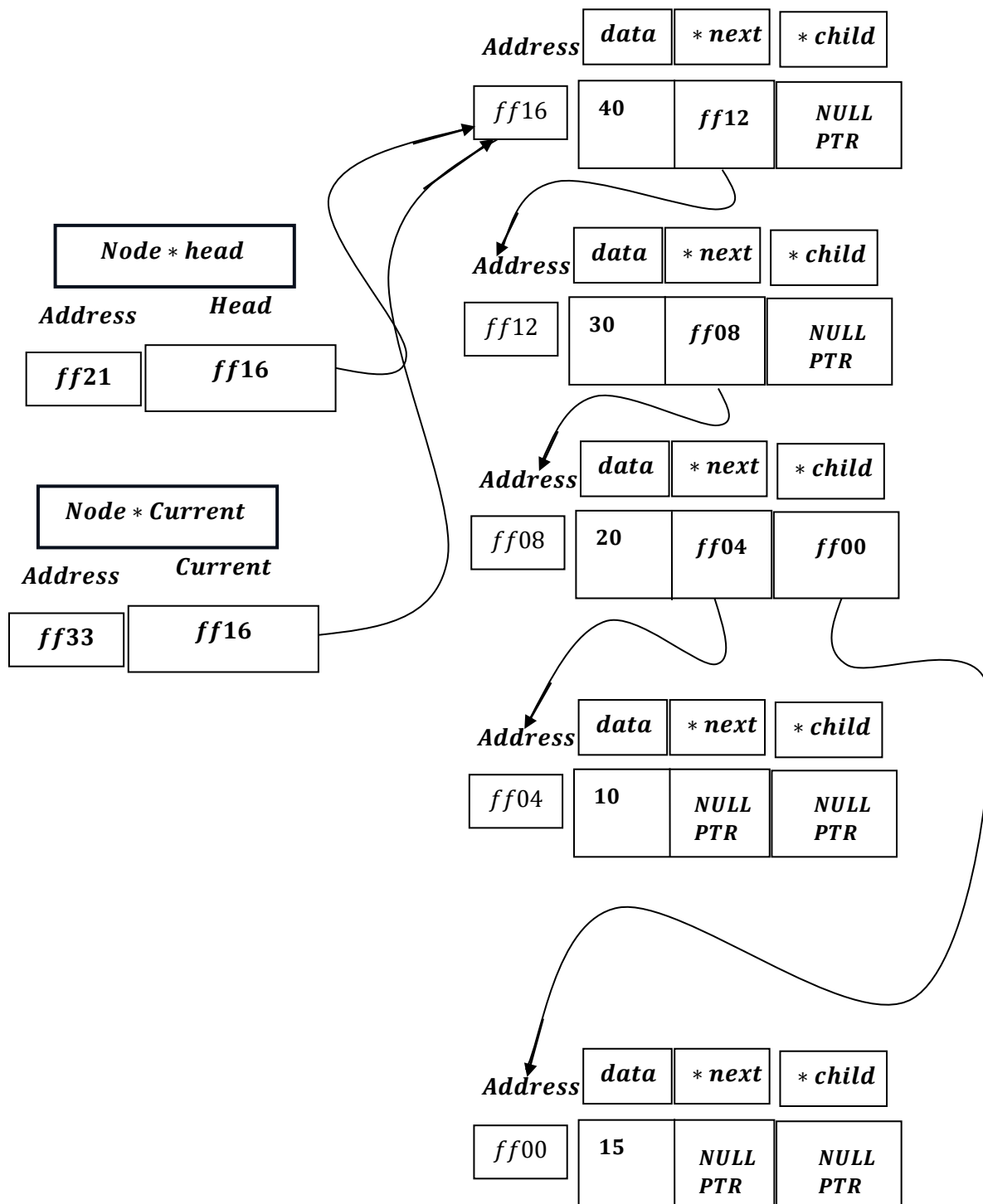
```
Node *temp = searchNode(head, target);
```

*∴ Node * temp = searchNode(head: ff16, target: 15);*

Now,

```
Node *searchNode(Node *current, int target)
```

∴ searchNode(Current = ff16, target: 15);



```
if (current == nullptr)
    return nullptr;
```

Current $\neq$ *NULLPTR*

```
if (current->data == target)
    return current;
```

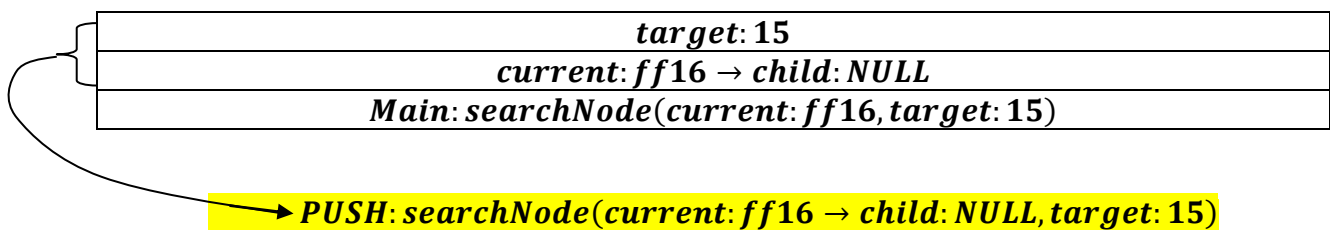
current $\rightarrow$ *data*: 40 $\neq$ *target*: 15

```
Node *foundInChild = searchNode(current->child, target);
```

current $\rightarrow$ *child* = *NULL* and *target* = 15.

$\therefore$ *Node* * *foundInChild* = *searchNode*(*current* $\rightarrow$ *child*: *NULL*, *target*: 15);

Recursion child - 1 (Push)



```
Node *searchNode(Node *current, int target)
```

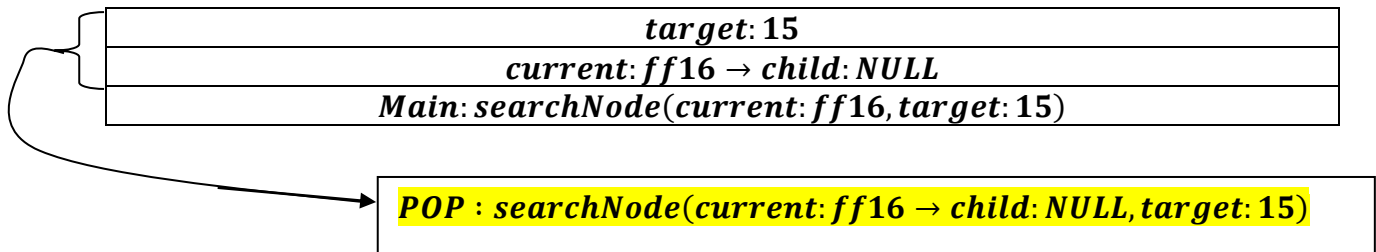
Assigning : *Node* * *searchNode*(*Node* * *current* = *NULL*, *int* *target* = 15)

```
if (current == nullptr)
    return nullptr;
```

Current = *NULLPTR* hence return *nullptr*;

As soon as it returns *NULLPTR* , then:

Recursion child – 1 (Pop)



As soon as it pop, *NULLPTR*, now :

```
Node *foundInChild = searchNode(current->child, target);
```

*Node * foundInChild = NULLPTR.*

```
if (foundInChild != nullptr)
{
    return foundInChild;
}
```

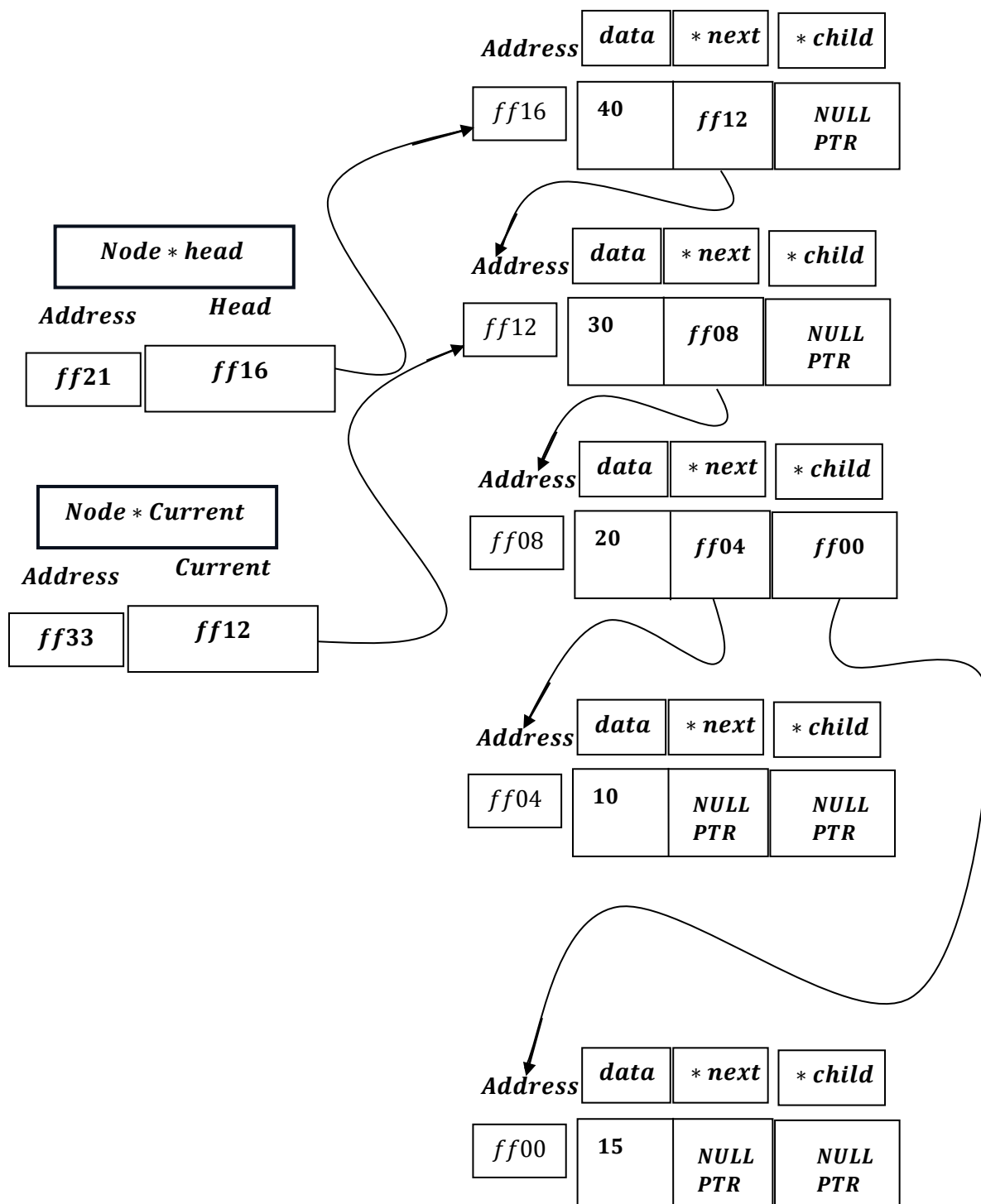
But *foundInChild = NULLPTR*; hence condition is false.

```
return searchNode(current->next, target);
```

searchNode(current → next: ff12, target: 15);

Recursion next push – 1

searchNode(current: ff12, target: 15)
Main: searchNode(current: ff16, target: 15)



```
if (current == nullptr)
    return nullptr;
```

Current $\neq$ NULLPTR

```
if (current->data == target)
    return current;
```

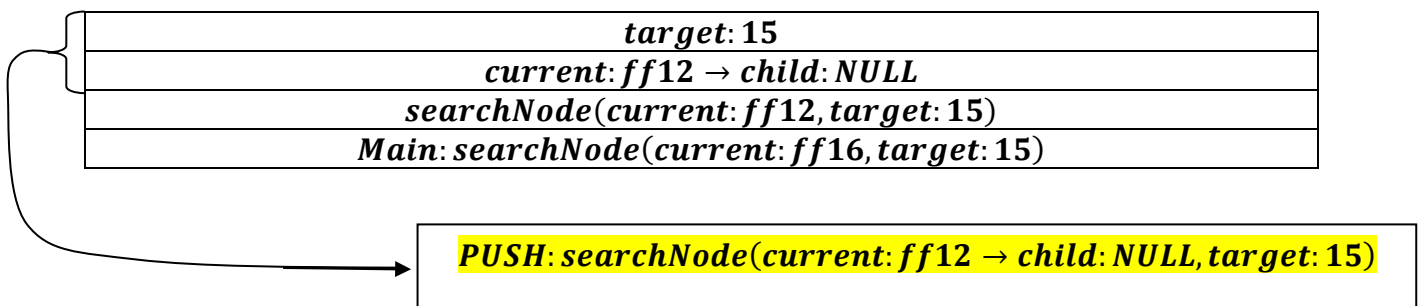
current $\rightarrow$ data: 30 $\neq$ target: 15

```
Node *foundInChild = searchNode(current->child, target);
```

current $\rightarrow$ child = NULL and target = 15.

$\therefore$ Node * foundInChild = searchNode(current $\rightarrow$ child: NULL, target: 15);

Recursion child – 2 (Push)



```
Node *searchNode(Node *current, int target)
```

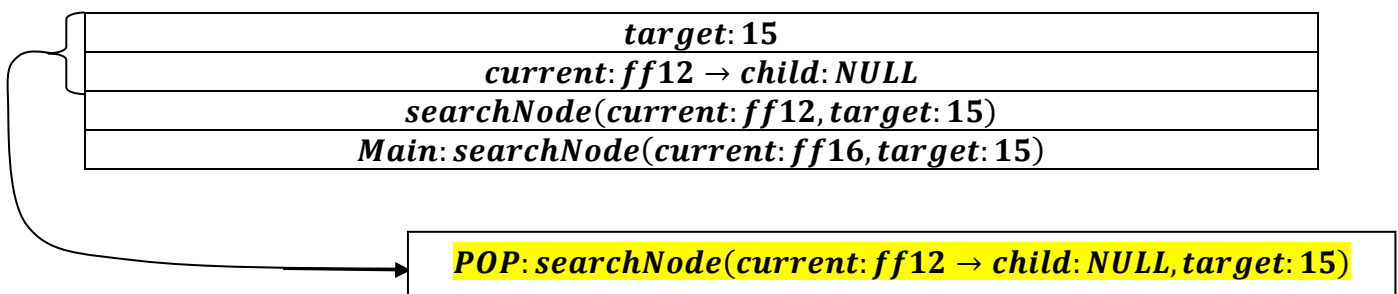
Assigning : Node * searchNode(Node * current = NULL, int target = 15)

```
if (current == nullptr)
    return nullptr;
```

Current = NULLPTR hence return nullptr;

As soon as it returns NULLPTR , then:

Recursion child – 2 (POP)



As soon as it pop, NULLPTR, now :

```
Node *foundInChild = searchNode(current->child, target);
```

Node * foundInChild = NULLPTR.

```
if (foundInChild != nullptr)
{
    return foundInChild;
}
```

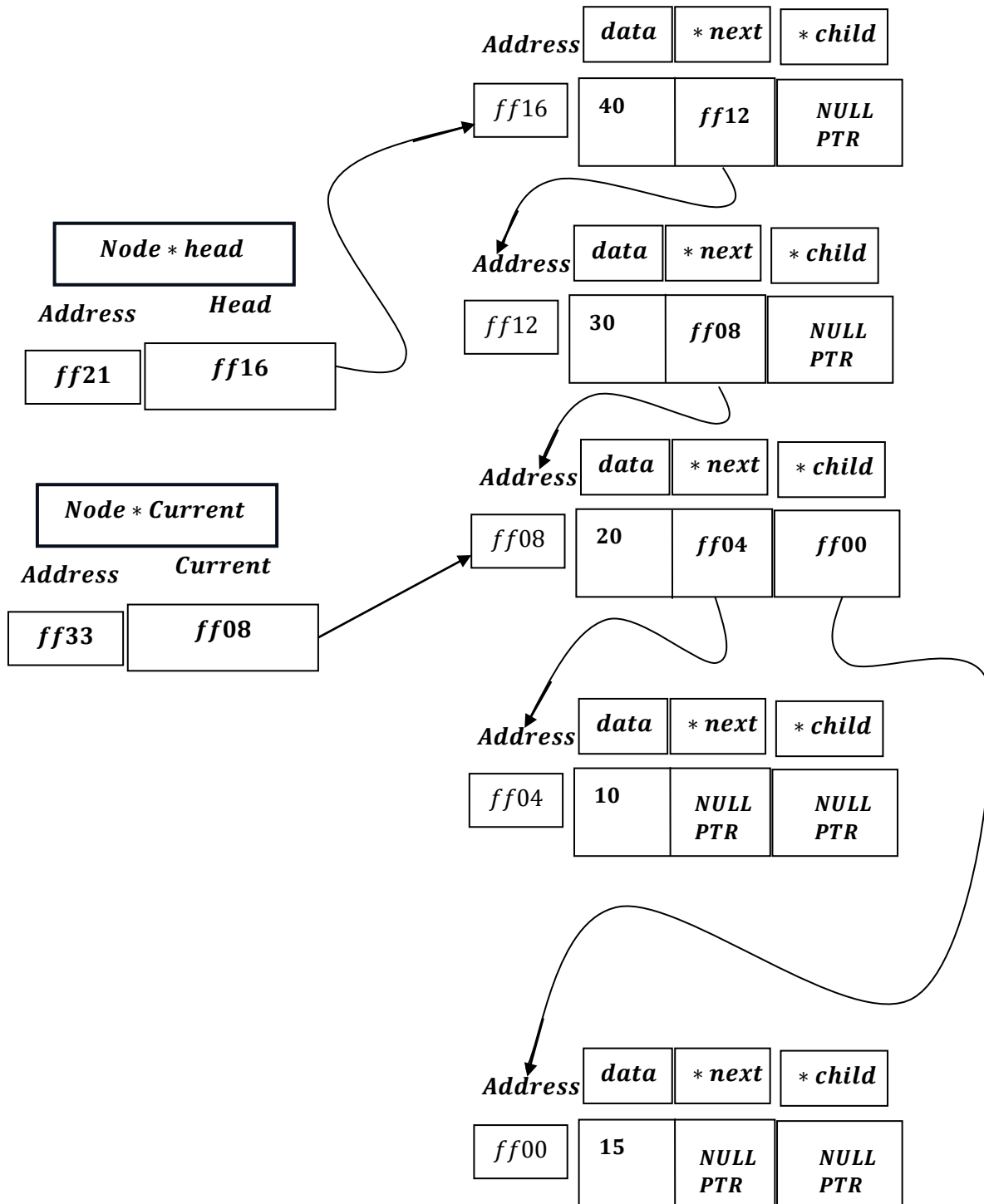
*But **foundInChild = NULLPTR**; hence condition is false.*

```
return searchNode(current->next, target);
```

searchNode(current → next: ff08, target: 15);

Recursion next push – 2

<i>searchNode(current: ff08, target: 15)</i>
<i>searchNode(current: ff12, target: 15)</i>
<i>Main: searchNode(current: ff16, target: 15)</i>



```
if (current == nullptr)
    return nullptr;
```

Current $\neq$ *NULLPTR*

```
if (current->data == target)
    return current;
```

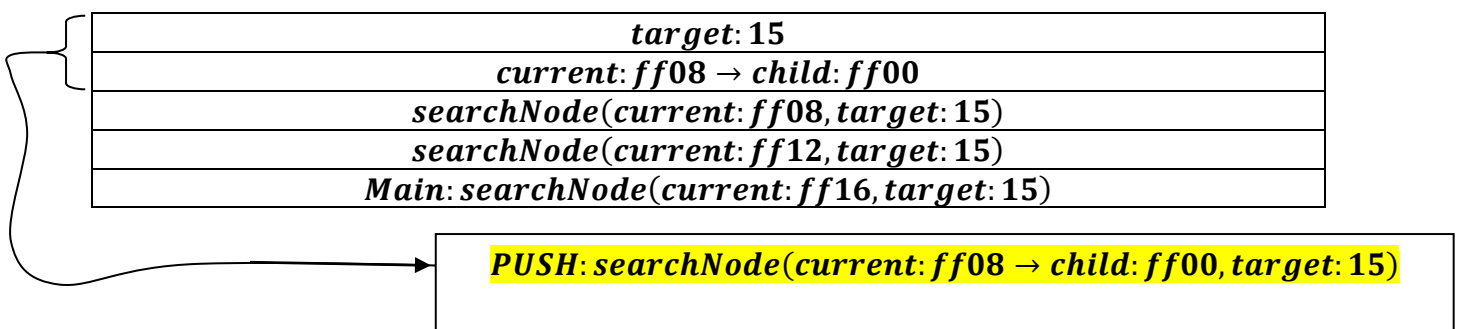
current $\rightarrow$ *data*: 20 $\neq$ *target*: 15

```
Node *foundInChild = searchNode(current->child, target);
```

current $\rightarrow$ *child* = *ff00* and *target* = 15.

$\therefore$ *Node* * *foundInChild*
= *searchNode*(*current*: *ff08* $\rightarrow$ *child*: *ff00*, *target*: 15);

Recursion child push – 3



```
Node *searchNode(Node *current, int target)
```

*Assigning : Node * searchNode(Node * current = ff00, int target = 15)*

```
if (current == nullptr)
    return nullptr;
```

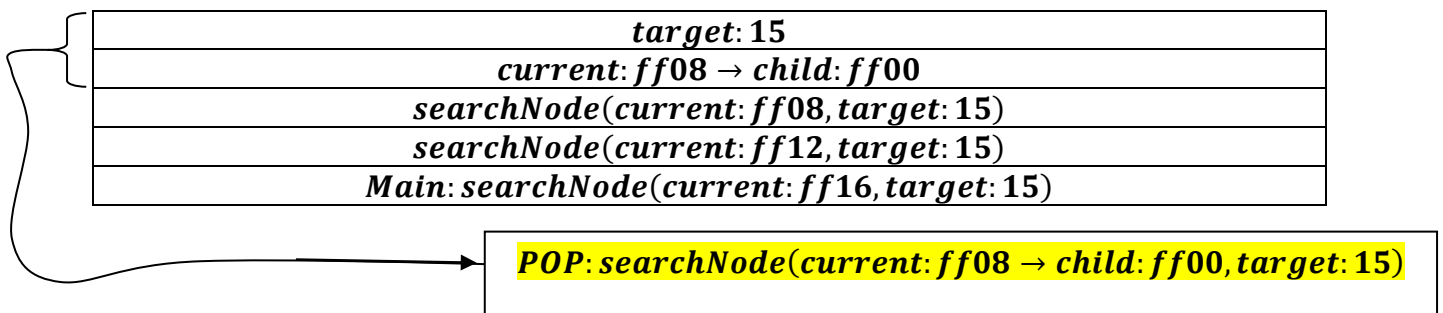
Current ≠ NULLPTR i.e. Current = ff00 hence condition failed.

```
if (current->data == target)
    return current;
```

current → data: 15 = Target = 15 , hence it is true , and return current: ff00.

As soon as it return current : ff00 .

Recursion child pop – 3



Very important: Thus end

searchNode(current: ff08 → child: ff00, target: 15) , not the whole function.

function : searchNode(current: ff08, target: 15) , still waiting for end.

As soon as it pop, ff00, now :

```
Node *foundInChild = searchNode(current->child, target);
```

Node * foundInChild = ff00.

```
if (foundInChild != nullptr)
{
    return foundInChild;
}
```

foundInChild: ff00 $\neq$ NULLPTR; hence condition is True:

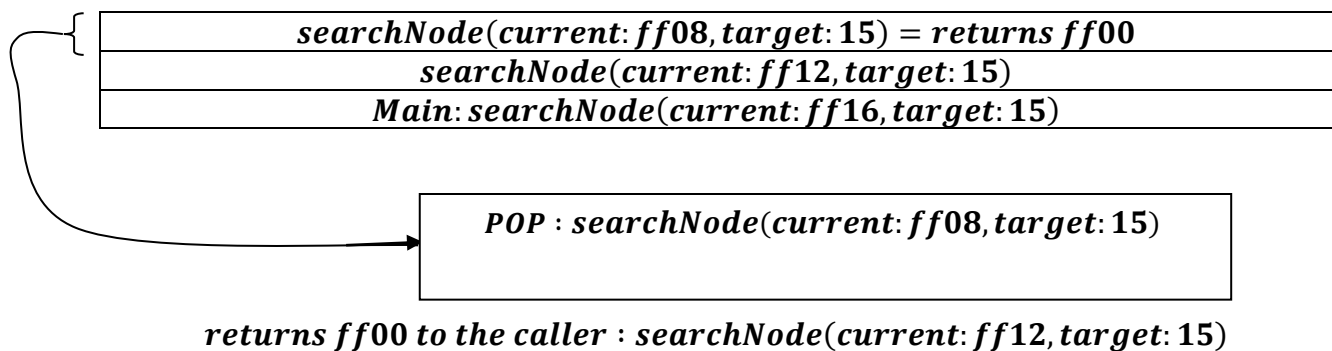
return foundInChild: ff00.

As it return foundInChild: ff00 :

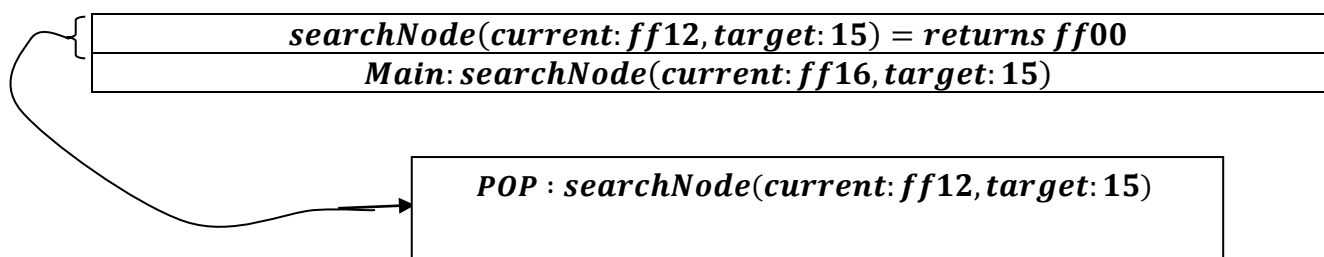
Thus , function: *searchNode(current: ff08, target: 15)* $\rightarrow$ ends here

.

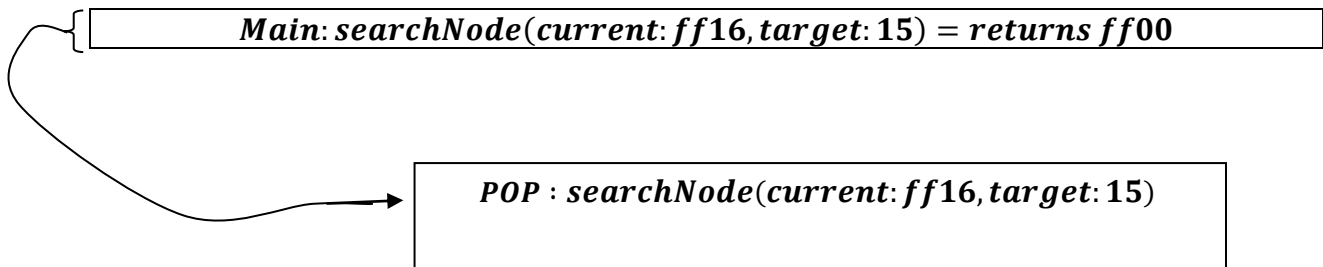
Recursion next pop – 2



Recursion next pop – 3



Recursion next pop – 4



Hence return : ff00.

```
Node *temp = searchNode(head, target);
```

*Node * temp = ff00.*

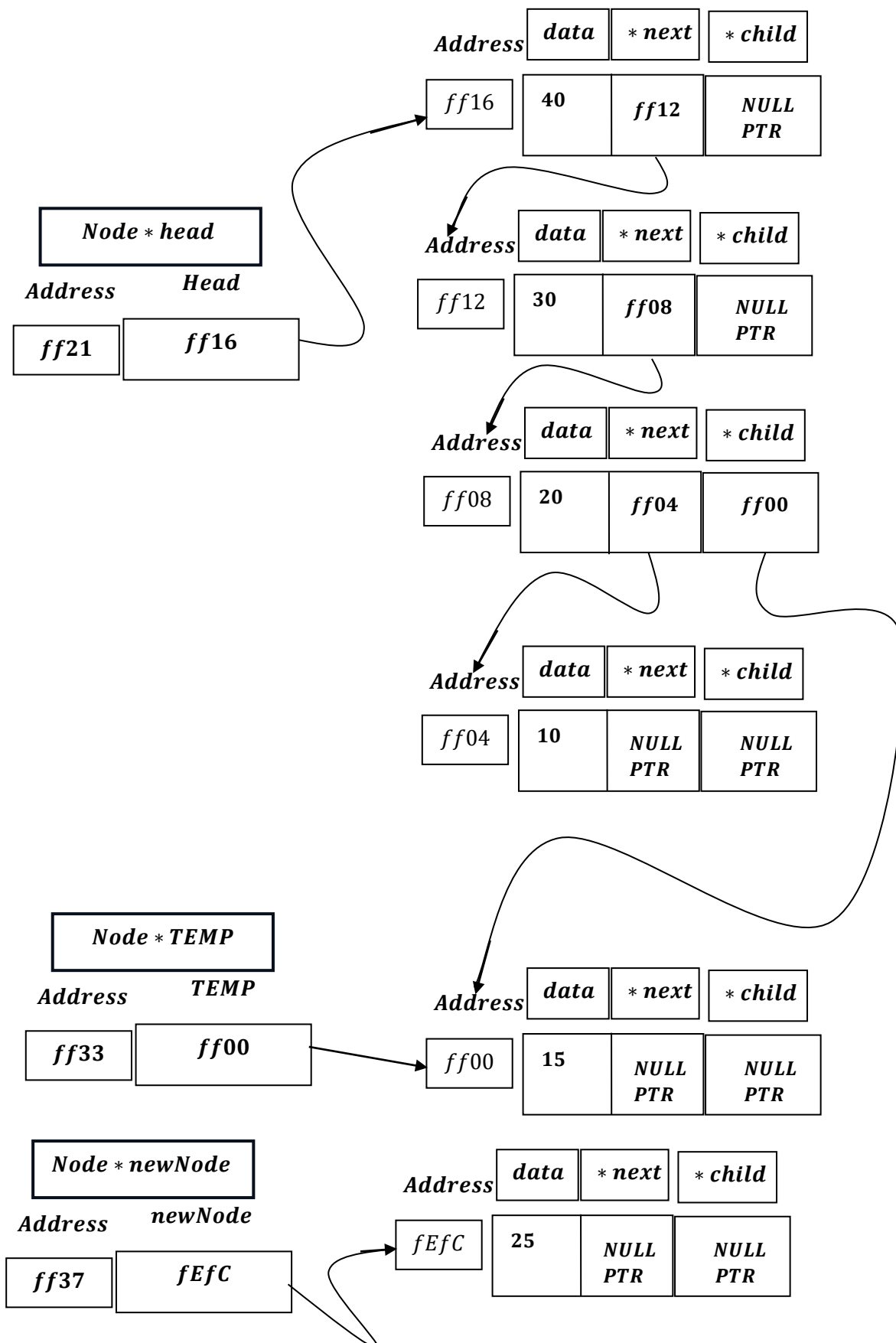
```
if (temp == nullptr)
{
    cout << "Element " << target << " not found anywhere
in the list." << endl;
    return;
}
```

Temp : ff00 ≠ NULLPTR, hence condition is false.

```
cout << "Enter data to insert: ";
cin >> data;
Node *newNode = createNode(data);
```

Enter data : 25.

```
Node *newNode = createNode(data);
```



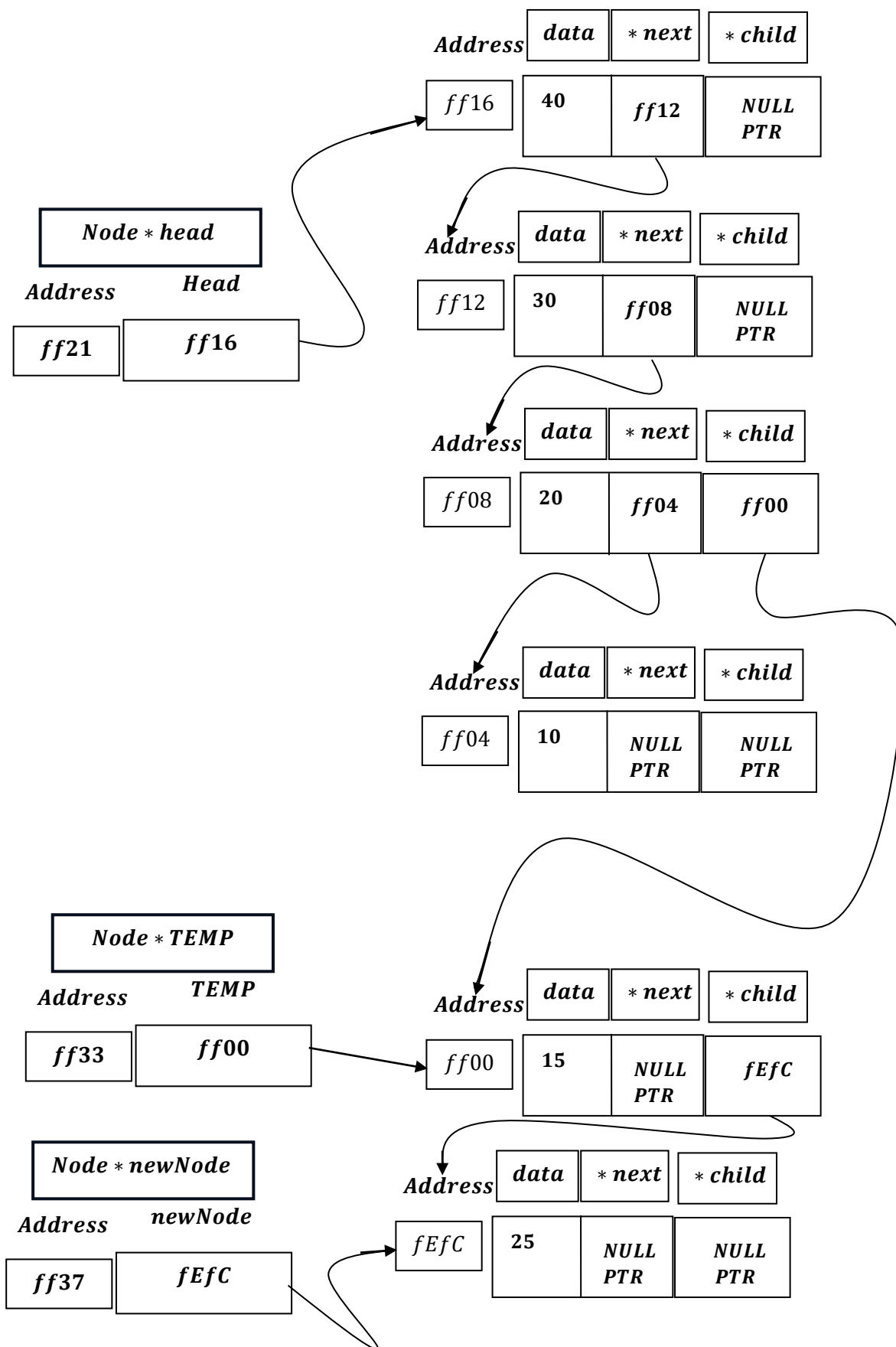
```
cout << "Insert as (1) Next Node or (2) Child Node? ";  
cin >> type;
```

Lets say type: 2 (Child)

```
else if (type == 2)  
{  
    newNode->next = temp->child;  
    temp->child = newNode;  
    cout << "Inserted " << data << " as CHILD of " <<  
target << endl;  
}
```

newNode → next = temp → child = NULLPTR

Temp → child = newNode = fEfC



Finished
